

Contents

1 5) Geometric Algebra / Dual Space formulation (this is where it gets elegant)

Exported on: 2/20/2026, 1:15:54 PM

1.1 You

Help me to understand the mathematical constraints of deformable surfaces made up out of equilateral hexagonal shapes of rigid material. No bending only the hinges angles can change.

Like FEM algorithms there must be some sort of tri-diagonal shaped matrix equation that defines the possible coordinates. I have already discussed some phase space ideas of allowed dynamic movement that only affects nearest neighbours. Please check if some Geometric Algebra formulation can solve this using the “Dual Space”.

1.2 ChatGPT

I'll formalize the kinematic constraints for a hinge-connected equilateral-hexagon lattice, show how they become sparse (banded) constraint Jacobians, and then sketch a clean Geometric Algebra / dual (Plücker) formulation for hinges and edge constraints.

Thought for 1m 4s

Model it as a bar-and-hinge / rigid-body constraint system

You have a polyhedral surface: each face is a rigid equilateral hexagon, and adjacent faces are connected by ideal hinges along shared edges (dihedral angle free; no in-face bending).

A clean way to write “all allowed coordinates” is:

unknowns = pose of each hexagon

constraints = shared-edge coincidence (two faces must literally share the same 3D edge segment)

That is exactly the same mathematical structure as multibody dynamics / rigid origami kinematics: nonlinear constraints + sparse Jacobian.

1) Variables: one rigid pose per hexagon

Let face i have pose (R_i, t_i) with

$$R_i \in SO(3)$$

$$t_i \in \mathbb{R}^3$$

Pick local vertex coordinates for a regular hexagon of side length s . A convenient choice is using circumradius s (for a regular hexagon, circumradius = side length):

$$u_k = s \begin{bmatrix} \cos(k\pi/3) \\ \sin(k\pi/3) \\ 0 \end{bmatrix}, \quad k = 0, \dots, 5$$

World vertex positions:

$$p_{i,k} = R_i u_k + t_i$$

So the full state is $x = \{R_i, t_i\}_{i=1}^F$ (6 DOF per face, with a careful parameterization for R_i , e.g. quaternions + unit constraint, or exponential coordinates).

2) Hinge constraint = “shared edge endpoints coincide”

If faces i and j share an edge, that edge corresponds to two vertices in each face. Say in face i it's (a, b) and in face j it's (c, d) with consistent ordering.

Then the constraints are simply:

$$p_{i,a} - p_{j,c} = 0, \quad p_{i,b} - p_{j,d} = 0$$

That's 6 scalar equations per shared edge, but crucially their Jacobian generically has rank 5, leaving exactly 1 relative DOF (rotation about the edge line) — i.e. a hinge.

Why rank 5 (intuition): the relative motion “rotate face j around the common edge line” produces zero instantaneous change in those endpoint-equality constraints.

3) The “FEM-like matrix” is the constraint Jacobian (sparse, graph-structured)

Collect all constraints into a vector $c(x) = 0$. Linearize:

$$c(x + \Delta x) \approx c(x) + J(x) \Delta x$$

Solve (Gauss-Newton / Newton):

$$J \Delta x = -c \text{ Sparsity / “tri-diagonal-ish”}$$

Each edge constraint touches only two faces (i, j) , so each constraint row has nonzeros only in the blocks for i and j . That means:

For a strip / chain of hexagons, the face adjacency graph is 1D $\rightarrow$ the block structure becomes block tridiagonal (your intuition).

For a 2D hex mesh, it's not tridiagonal, but it is still very sparse: the pattern is the mesh adjacency graph.

If you enforce constraints with Lagrange multipliers (typical in rigid-body solvers), you get the classic KKT system:

$$\begin{bmatrix} H & J^\top \\ J & 0 \end{bmatrix} \begin{bmatrix} \Delta x \\ \lambda \end{bmatrix} = \begin{bmatrix} -g \\ -c \end{bmatrix}$$

Sparse indefinite, solved by sparse LDL^T / SQP methods. This is the rigid-kinematics analog of FEM's sparse systems.

4) “Nearest-neighbor only” is true locally — but cycles add global compatibility

If the adjacency graph has no cycles (a tree), then:

choose a root face pose (R_0, t_0)

choose hinge angles on edges

all other face poses are determined by propagating across edges

But a real 2D tiling has many cycles, and those impose closure constraints (“holonomy”): the product of relative transforms around any loop must be identity.

That’s the key reason the motion space isn’t purely local in a 2D lattice: loops couple the angles.

5) Geometric Algebra / Dual Space formulation (this is where it gets elegant)

For hinges, Projective Geometric Algebra (PGA) (or Conformal GA if you need spheres etc.) is a natural fit. The “dual space” language usually means: represent lines as dual bivectors (intersection of planes) and use motors for rigid motions.

Core objects (PGA view)

A rigid motion is a motor X_i (rot+trans in one multivector).

A line (hinge axis) is a bivector L . In 3D PGA, a line is naturally represented and its dual form corresponds to “intersection of two planes”.

Rotation about a line = exponential of a line bivector

A pure rotation by angle θ about line L is:

$$M(\theta) = \exp\left(-\frac{\theta}{2} \hat{L}\right)$$

($\hat{L}$ normalized appropriately.)

Express the hinge constraint between two faces as a relative motor

Let $L_{i,e}$ be the hinge line in face i ’s local coordinates (the shared edge line in that rigid hexagon’s frame).

World hinge line from face i ’s current pose:

$$L_e = X_i L_{i,e} X_i^{-1}$$

Relative pose between faces:

$$X_{ji} = X_j X_i^{-1}$$

Hinge condition: the relative motor must be a rotation about the shared edge line:

$$X_j X_i^{-1} = \exp\left(-\frac{\theta_e}{2} \hat{L}_e\right)$$

This single equation packs the same content as “edge endpoints coincide + 1 DOF hinge”, but in a form that’s often numerically and conceptually cleaner.

Loop (cycle) constraints become a product of exponentials

For any cycle C of edges $e_1, \dots, e_m$:

$$\prod_{k=1}^m \exp\left(-\frac{\theta_{e_k}}{2} \hat{L}_{e_k}\right) = 1$$

That’s the GA version of “closure”.

This is the precise place where a ‘dual space’/connection viewpoint helps: hinge axes are lines (dual bivectors), and you’re solving a discrete compatibility problem on a graph (very close to a gauge/connection formulation).

6) Practical takeaway: what you can actually solve

Two common choices of unknowns:

A) Solve for face poses directly (multibody style)

Unknowns: X_i (or R_i, t_i) Constraints: shared-edge coincidence Method: Newton / Gauss-Newton with sparse J (graph sparsity).

B) Solve for hinge angles with loop closure (connection style)

Unknowns: θ_e on edges + one root pose Constraints: cycle products = identity Method: pick a spanning tree (local propagation), then add loop constraints on non-tree edges $\rightarrow$ sparse nonlinear least squares.

If your goal is “phase space of allowed motion”:

tree edges give local DOF

cycle constraints cut that down to the actual mechanism space (possibly still floppy, but globally consistent)

7) A quick DOF sanity check (why mechanisms exist / disappear)

Let $F = \# \text{faces}$, $E = \# \text{shared edges}$. Naively:

DOF: $6F$

constraints: $\approx 5E$ independent (hinge removes 5 relative DOF)

Then subtract global rigid motion (6) and add back constraint dependencies (cycles) appropriately. On a 2D lattice there are many dependencies, so you can indeed get nontrivial mechanisms (like rigid origami).

If you want a sharp next step (no extra questions required)

If you tell me your topology (e.g. “finite $m \times n$ patch of hexes, boundary free” vs “periodic torus”), I can write the exact:

constraint count,

expected mechanism dimension (generic Maxwell-style),

and the explicit sparse block structure of J / KKT .

But even without that, the key mathematical constraints are exactly:

rigid face embedding $p_{i,k} = R_i u_k + t_i$

shared-edge coincidence constraints (sparse, nearest-neighbor)

cycle closure / holonomy constraints (global, kills inconsistent local motions)

optional GA form: relative motors are exponentials of dual line bivectors.